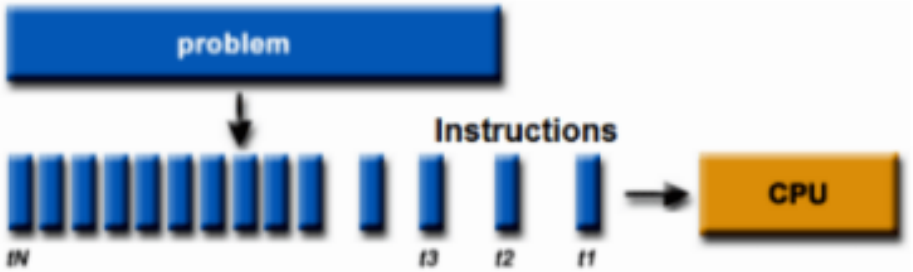
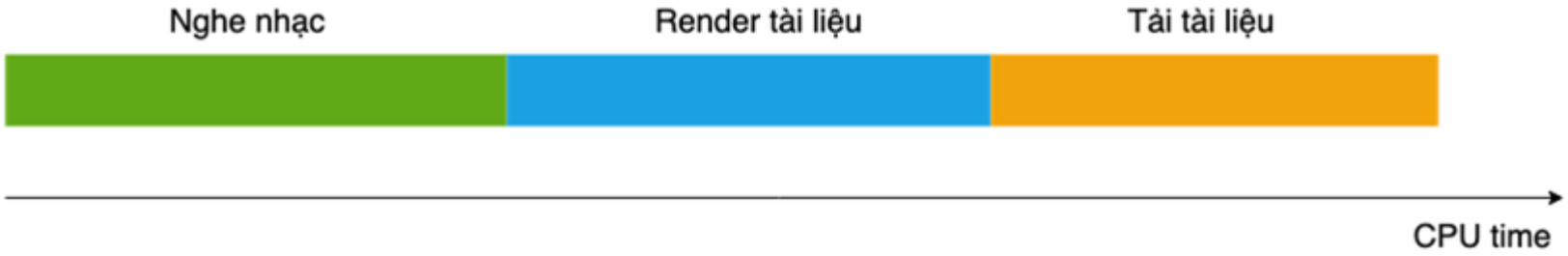
CHƯƠNG 4:
TÍNH TOÁN BẤT ĐỒNG BỘ

GIỚI THIỆU

- **Xử lý tuần tự**
  - Xử lý 1 tác vụ (task) trong một khoảng thời gian;
  - Các task sẽ được thực thi theo thứ tự.

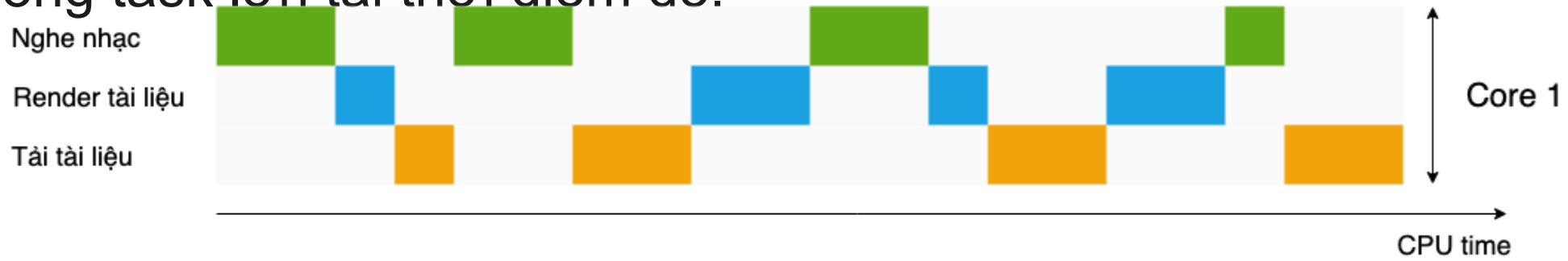




GIỚI THIỆU

➢ Xử lý đồng thời

- HĐH quản lý chương trình, mỗi chương trình ~ process (tiến trình) → cấp process id (PID) để HĐH quản lý.
- CPU core chia các task của process thành các task nhỏ hơn + sắp xếp xen kẽ nhau → tận dụng thời gian rảnh của task này đi thực hiện task khác.
- → Máy tính xử lý nhiều việc cùng lúc tại cùng thời điểm, tuy nhiên, bên dưới CPU core chỉ có thể thực thi một task nhỏ trong task lớn tai thời điểm đó.



GIỚI THIỆU

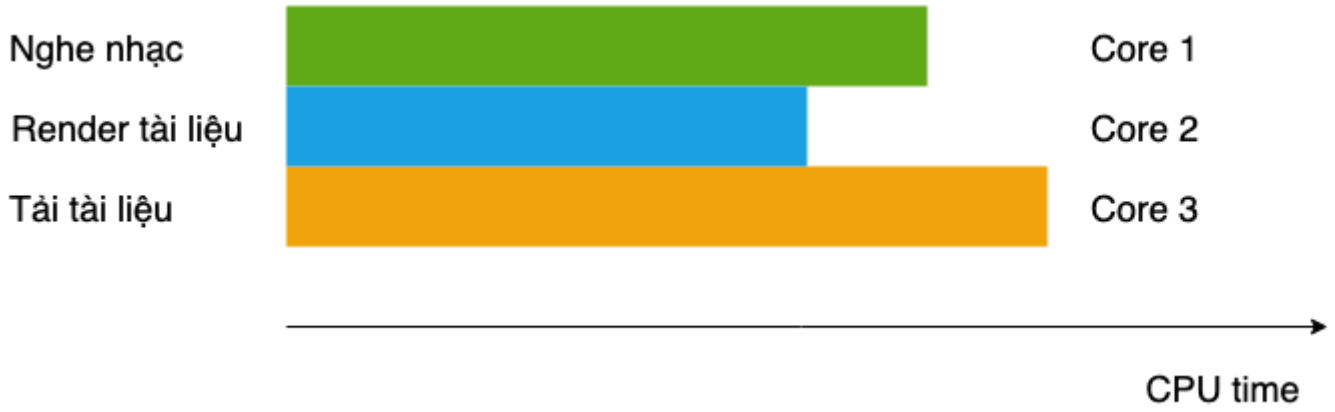
- **Xử lý đồng thời**

  - Phân chia, điều phối nhiều tác vụ (task) khác nhau trong cùng một khoảng thời gian.
  - Tại 1 thời điểm, chỉ xử lý 1 task.



GIỚI THIỆU

- **Xử lý song song**
  - Nhiều task khác nhau được xử lý trong cùng 1 thời điểm;
  - Các task hoàn toàn độc lập nhau.
  - Chỉ có thể thực hiện trên máy có số nhân (core) lớn hơn 1.



GIỚI THIỆU

- **Xử lý song song**

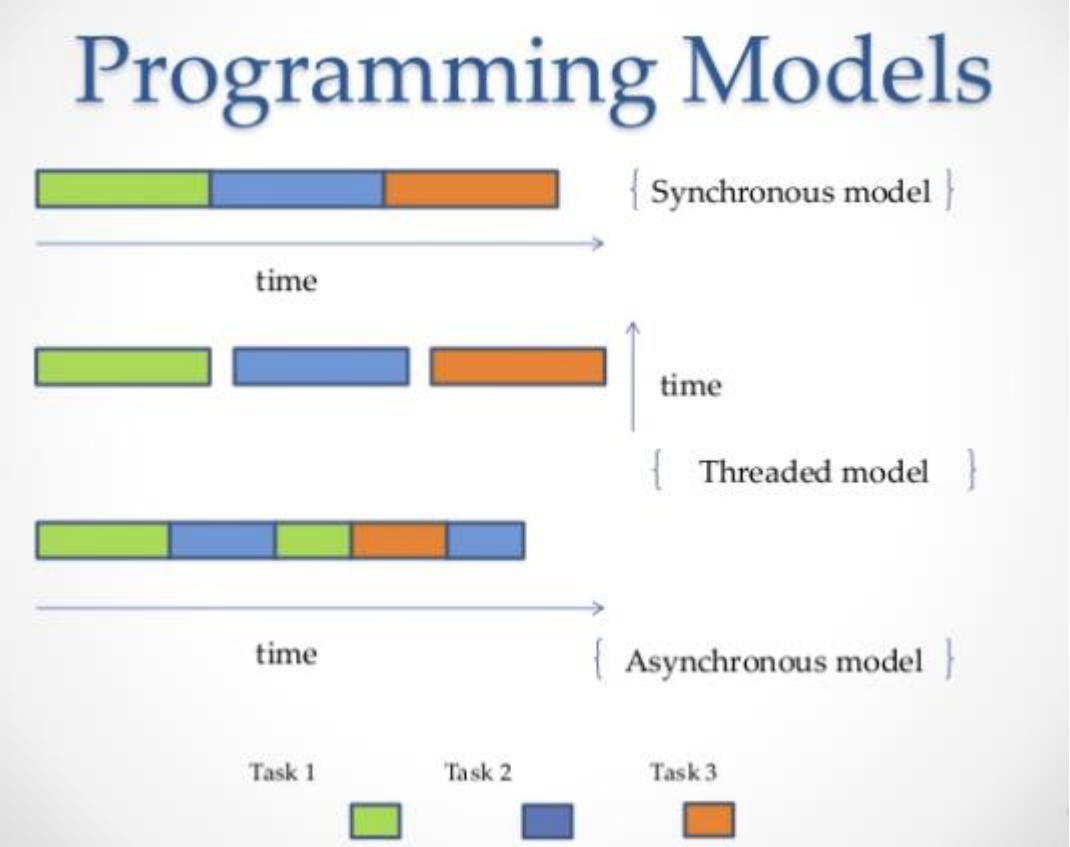
  Trên thực tế, trên mỗi nhân của CPU vẫn xảy ra quá trình xử lý đồng thời, với điều kiện là tại một thời điểm không xảy ra việc xử lý cùng một task trên hai nhân CPU khác nhau.



GIỚI THIỆU

- **Bất đồng bộ**

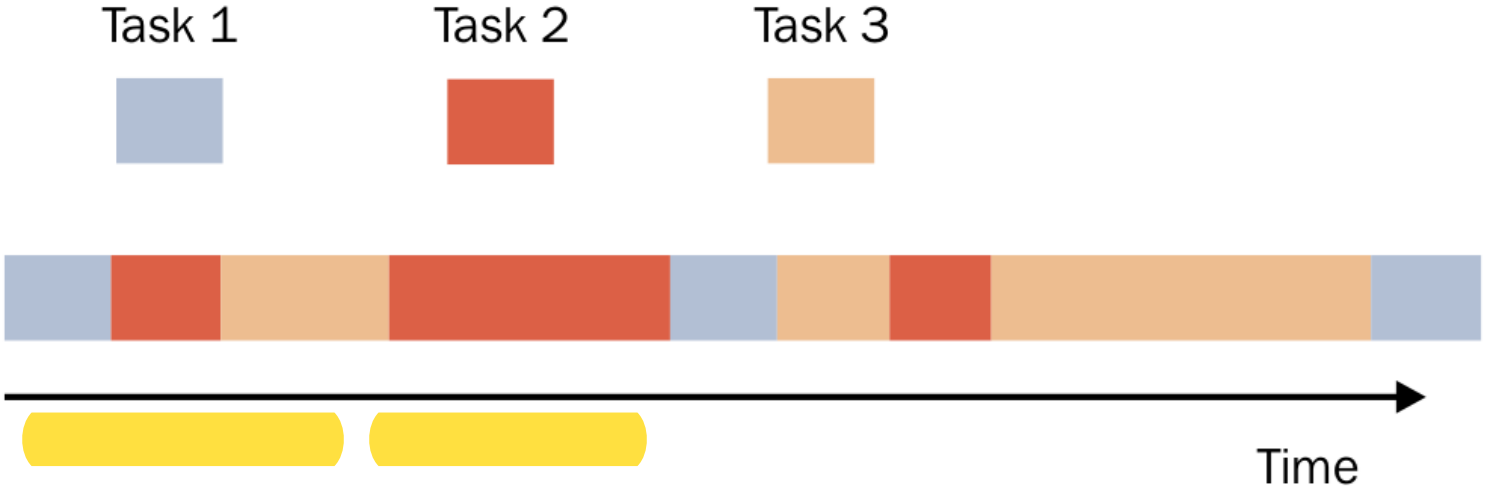
  - Cho phép các task được xử lý không theo thứ tự (có thể chuyển sang task khác trước khi task trước đó hoàn thành).
  - Có thể giải quyết nhiều yêu cầu đồng thời.
  - Hoàn thành trong khoảng thời gian ngắn hơn.
  - VD: Đi thi, bãi giữ xe, ...



GIỚI THIỆU

- **Mô hình bất đồng bộ thực thi đồng thời**

- Trong một khoảng thời gian, các task thực hiện xen kẽ nhau;
- Nằm trong một luồng điều khiển duy nhất → khi 1 task đang thực thi thì các task khác không thực thi (việc thực thi của 1 task có thể bị tạm dừng và sau đó tiếp tục lại);
- Sử dụng một luồng điều khiển chính duy nhất, cả trong hệ thống đơn bộ xử lý (uniprocessor systems) và hệ thống đa bộ xử lý (multiprocessor systems).



SỬ DỤNG MODULE CONCURRENT.FUTURE

- **Module concurrent.future**

- Được phát hành đi kèm phiên bản Python 3.2;
- Là một high-level module.
- Cung cấp giao diện cấp cao để thực thi các lệnh gọi bất đồng bộ.
- Lập trình đồng thời (concurrent programming): xử lý task đa luồng hoặc tận dụng tính đa lõi của những CPU đời mới một cách dễ dàng và nhanh chóng.

SỬ DỤNG MODULE CONCURRENT.FUTURE

- **Module concurrent.future**

 Dựa trên hai module khác của Python đó là threading và multiprocessing.

 Việc **thực thi bất đồng bộ** có thể được thực hiện với các **luồng** (sử dụng **ThreadPoolExecutor**) hoặc các **tiến trình riêng biệt** (sử dụng **ProcessPoolExecutor**) → cả hai đều triển khai cùng một giao diện, được định nghĩa bởi lớp trừu tượng Executor.

SỬ DỤNG MODULE CONCURRENT.FUTURE

- **Đối tượng thực thi (Executor Objects)**

 concurrent.futures.Executor: lớp trừu tượng cung cấp các phương thức để thực hiện gọi bất đồng bộ, không nên được sử dụng trực tiếp mà thông qua các lớp con cụ thể của nó.

- submit (function ,argument)
- map (function,argument)
- shutdown (Wait = True)

 ThreadPoolExecutor: là một lớp con Executor sử dụng một nhóm các luồng để thực hiện các lệnh gọi không đồng bộ.

 ProcessPoolExecutor: là một lớp con Executor sử dụng một nhóm các tiến trình để thực hiện các lệnh gọi không đồng bộ.

 Chi tiết xem tại: https://docs.python.org/3/library/concurrent.futures.html

SỬ DỤNG MODULE CONCURRENT.FUTURE

- **Đối tượng tương lai (Future Objects)**

 Lớp Future đóng gói việc thực thi bất đồng bộ của một đối tượng có thể được gọi. Các instances trong Future được tạo bởi Executor.submit().

 concurrent.futures.Future: đóng gói việc thực thi không đồng bộ của một đối tượng có thể được gọi. Các instances trong Future được tạo bởi Executor.submit() và không được tạo trực tiếp ngoại trừ để thử nghiệm.

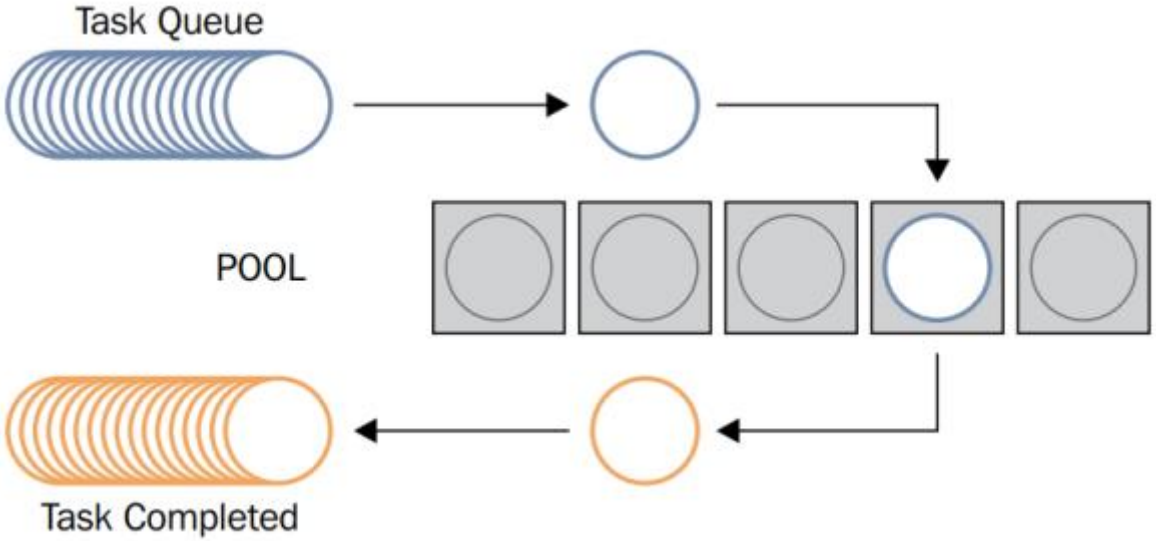
- Cancel(), Cancelled()
- Running(), done(), result(timeout=None)
- exception(timeout=None), add_done_callback(fn), ...

 Chi tiết xem tại: https://docs.python.org/3/library/concurrent.futures.html

SỬ DỤNG MODULE CONCURRENT.FUTURE

- **Xử lý nhóm tiến trình và luồng**

 Nhóm tiến trình hoặc luồng (nhóm được gọi): biểu thị trình quản lý phần mềm được sử dụng để tối ưu hóa và đơn giản hóa việc sử dụng luồng và/hoặc tiến trình trong một chương trình.



SỬ DỤNG MODULE CONCURRENT.FUTURE

- **Xử lý nhóm tiến trình và luồng**

 Một nhóm tiến trình/luồng duy trì nhiều tiến trình/luồng chờ các tác vụ được chương trình giám sát phân bổ để thực thi đồng thời.

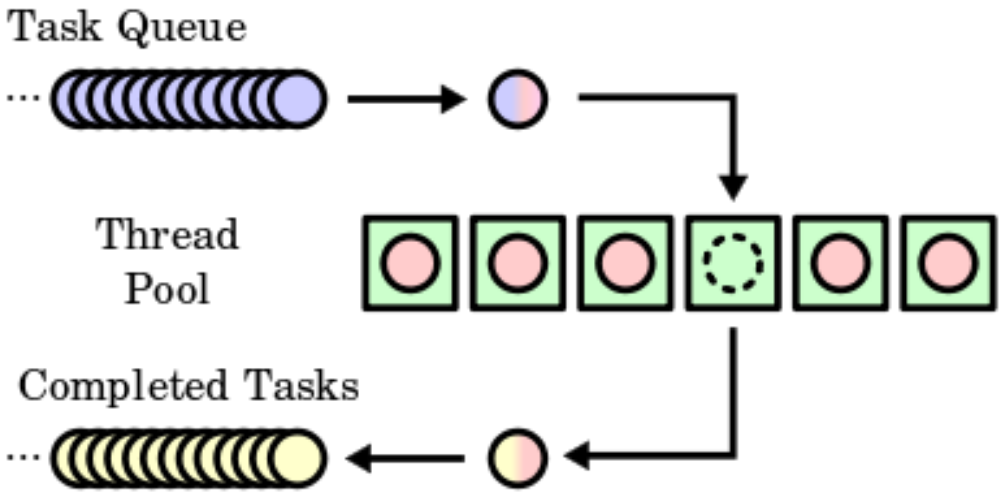
 Tăng hiệu suất và tránh độ trễ (lantency) khi thực thi do việc tạo và hủy tiến trình/ luồng thường xuyên cho các tác vụ có thời gian sử dụng ngắn.

 Số lượng tiến trình/ luồng được điều chỉnh theo tài nguyên có sẵn.

SỬ DỤNG MODULE CONCURRENT.FUTURE

- **Xử lý nhóm tiến trình và luồng**

 Reuse: một tiến trình/ luồng được sử dụng nhiều lần cho các tác vụ khác nhau trong vòng đời của nó.



QUẢN LÝ VÒNG LẶP SỰ KIỆN VỚI ASYNCIO

- **Sự kiện (Event)** là một hành động diễn ra khi người dùng tương tác với một chương trình.
  - VD: gõ phím/ click chuột là 1 sự kiện.
- **Trong một hệ thống tính toán**
  - Thực thể (Entity) có thể tạo sự kiện → **nguồn sự kiện (Event source)**;
  - Thực thể đàm phán để quản lý sự kiện → **trình xử lý sự kiện (Event handler);**
  - → Là 1 callback, được gọi sau khi một sự kiện (event) xảy ra.
  - → Lập trình viên viết mã cho callback này để thực hiện các hành động khi sự kiện xảy ra.
  - Thực thể thứ 3: **vòng lặp sự kiện (Event loop)**

QUẢN LÝ VÒNG LẶP SỰ KIỆN VỚI ASYNCIO

- **Vòng lặp sự kiện (Event loop)**

 Là một cấu trúc lập trình hoặc mẫu thiết kế chờ và gửi các sự kiện hoặc thông báo trong một chương trình.

 Hoạt động bằng cách đưa ra yêu cầu tới một số "nhà cung cấp sự kiện" bên trong hoặc bên ngoài, sau đó gọi trình xử lý sự kiện có liên quan.

 Hoạt động theo chu kỳ trong toàn bộ quá trình thực thi chương trình.

 Theo dõi các sự kiện đã xảy ra trong cấu trúc dữ liệu để xếp hàng và sau đó xử lý từng sự kiện một bằng cách gọi trình xử lý sự kiện nếu luồng chính rảnh.

QUẢN LÝ VÒNG LẶP SỰ KIỆN VỚI ASYNCIO

- **Module Python Asyncio**

 Là thư viện để code các tác vụ đồng thời (cocurrent) bằng cách sử dụng cú pháp async/await.

 Được sử dụng làm nền tảng cho nhiều framework bất đồng bộ trong Python: cung cấp mạng và máy chủ web hiệu suất cao, thư viện kết nối cơ sở dữ liệu, hàng đợi tác vụ phân tán, …

 Cung cấp các phương tiện để quản lý các sự kiện, coroutine, tác vụ và luồng, đồng bộ hóa, ...

 Chi tiết xem tại: https://docs.python.org/3/library/asyncio.html.

QUẢN LÝ VÒNG LẶP SỰ KIỆN VỚI ASYNCIO

- **Module Python Asyncio**: các thành phần chính gồm:

- **Event loop**: cho phép một vòng lặp sự kiện duy nhất cho mỗi tiến trình.

- **Coroutines**: sự tổng quát hóa của khái niệm subroutine (chương trình con ~ đồng nghĩa với hàm)

→ Một coroutine có thể bị treo trong quá trình thực thi để chờ xử lý bên ngoài (một số thủ tục trong I/O) và quay trở lại từ điểm đã dừng khi quá trình xử lý bên ngoài hoàn tất.

- **Futures**: xác định đối tượng Future, đại diện cho các giá trị bất đồng bộ được trả về từ các tác vụ.

→ Vd module concurrent.futures đại diện cho một tính toán vẫn chưa được hoàn thành.

- **Tasks**: là một lớp con của Asyncio được sử dụng để đóng gói và quản lý các coroutine ở chế độ song song.

XỬ LÝ TÁC VỤ ĐỒNG THỜI VỚI ASYNCIO

- Một chương trình dài và phức tạp → chia thành các chương trình con, mỗi chương trình thực hiện một nhiệm vụ cụ thể.
- Chương trình con chỉ thực thi khi có yêu cầu từ chương trình chính, **không thể thực thi độc lập.**
- Chương trình chính chịu trách nhiệm điều phối, sử dụng các chương trình con.
- → coroutine là sự tổng quát hoá chương trình con.

XỬ LÝ TÁC VỤ ĐỒNG THỜI VỚI ASYNCIO

- **Xử lý tác vụ đồng thời**: thực hiện **nhiều tác vụ bất đồng bộ đồng thời trên một luồng đơn** → cho phép chương trình thực hiện nhiều tác vụ mà không bị chặn bởi các hoạt động đồng bộ.
- Asyncio cung cấp **cơ chế** để **quản lý và thực hiện** các tác vụ **bất đồng bộ** bằng cách **sử dụng các coroutine**.
- Trong asyncio, các **coroutine** là các **hàm bất đồng bộ** được đánh dấu bằng từ khóa "**async**".
- **Các coroutine** có thể **chạy đồng thời trên một luồng đơn**, mỗi coroutine đang chờ đợi kết quả của một tác vụ bất đồng bộ khác.

XỬ LÝ TÁC VỤ ĐỒNG THỜI VỚI ASYNCIO

- Coroutine thực hiện **1 bước tính toán** duy nhất.
- Các coroutine tự **liên kết với nhau** để tạo thành một đường ống + **không có hàm giám sát** nào chịu trách nhiệm gọi chúng theo một thứ tự cụ thể.
- → Cho phép các **tác vụ đang chạy độc lập chạy đồng thời với các tác vụ khác** trên cùng một vòng lặp sự kiện.
- Coroutine có thể tạm dừng và chuyển luồng điều khiển cho một coroutine khác → các coroutine có thể chuyển các giá trị cùng với luồng điều khiển sang một coroutine khác → "yielding a value" được sử dụng để mô tả việc tạo và chuyển một giá trị cho coroutine nhận điều khiển.

LƯU Ý

- Sử dụng module cocurrent.futures
- Quản lý vòng lặp sự kiện với Asyncio
- Xử lý tác vụ đồng thời với Asyncio



